Issues

HelixCode — Open Issues Tracker

> Per Constitution §11.4.15 (Item-status tracking) + §11.4.16 (Item-type tracking) + §11.4.19 (Fixed-document column-alignment) + CONST-057 (Type-aware closure vocabulary) + CONST-058 (Reopened-source attribution).
>
> **Authoritative resumption ledger**: `docs/CONTINUATION.md` (CONST-044). This file complements it with item-level granularity for currently-open work.
>
> **Status vocabulary** (closed set): `Queued` | `In progress` | `Ready for testing` | `In testing` | `Reopened` | `Fixed/Implemented/Completed (→ Fixed.md)`
>
> **Type vocabulary** (closed set): `Bug` | `Feature` | `Task`

Prefix convention

Round 189 (2026-05-19) introduced per-project / per-submodule ID prefixes replacing the legacy `ISSUE-NNN` flat namespace. New items MUST use the prefix matching their scope; cross-cutting items affecting two or more submodules (or any meta-repo concern) live under `HXC`. Numeric portion is per-prefix (each prefix starts at `001`). Legacy `ISSUE-NNN` IDs renamed forward-only per CONST-043; historical close-out narratives in `docs/CONTINUATION.md` preserve original IDs verbatim, and `docs/Fixed.md` annotates each closure with `(ex-ISSUE-NNN)` for git-history traceability.

Prefix	Scope	Source
HXC	HelixCode root project (project-wide, multi-submodule, governance, infrastructure)	this repo
HXA	HelixAgent submodule	`submodules/helix_agent` (when present) / `helix_agent/` tree

Prefix	Scope	Source
HXM	HelixMemory submodule	`submodules/helix_memory`
HXL	HelixLLM submodule	`submodules/helix_llm`
HXQ	HelixQA submodule	`submodules/helix_qa` (`helix_qa/`)
HXS	HelixSpecifier submodule	`submodules/helix_specifier`
HXO	HelixOrchestrator (= LLMOrchestrator) submodule	`submodules/llm_orchestrator`
HXV	HelixVerifier (= LLMsVerifier) submodule	`submodules/llms_verifier`
HXD	HelixDocProcessor (= DocProcessor) submodule	`submodules/doc_processor`
HXI	HelixI18n (when added)	tba
PLN	Planning submodule (vasic-digital)	`submodules/planning`
VEN	VisionEngine submodule (HelixDevelopment)	`submodules/vision_engine`
SLF	SelfImprove submodule (vasic-digital)	`submodules/self_improve`
STO	Storage submodule (vasic-digital)	`submodules/storage`
OPS	LLMOps submodule (vasic-digital)	`submodules/llm_ops`
VDB	VectorDB submodule (vasic-digital)	`submodules/vector_db`
OBS	Observability submodule (vasic-digital)	`submodules/observability`
MCP		

Prefix	Scope	Source
	MCP_Module submodule (vasic-digital)	`submodules/mcp_module`
MSG	Messaging submodule (vasic-digital)	`submodules/messaging`
MDW	Middleware submodule (vasic-digital)	`submodules/middleware`
PLG	Plugins submodule (vasic-digital)	`submodules/plugins`
STR	Streaming submodule (vasic-digital)	`submodules/streaming`
WAT	Watcher submodule (vasic-digital)	`submodules/watcher`
CNV	conversation submodule (vasic-digital)	`submodules/conversation`
AUT	Auth submodule (vasic-digital)	`submodules/auth`
LZY	Lazy submodule (vasic-digital)	`submodules/lazy`
ATP	AutoTemp submodule (vasic-digital)	`submodules/auto_temp`
PLI	PliniusCommon submodule (vasic-digital)	`submodules/plinius_common`
CHL	challenges submodule (vasic-digital)	`challenges/`
CNT	containers submodule	`containers/`
SEC	security submodule	`security/`
PAN	panoptic submodule	`panoptic/`

For submodules not listed above, default to the first 3 letters of the submodule name, uppercase (e.g. `Watcher` → `WAT`). Document the new prefix in this table on first use.

Legacy → new mapping (round 189)

Old ID	New ID	Scope rationale
ISSUE-001	VEN-001	VisionEngine `helix-gitlab` URL
ISSUE-002	VEN-002	VisionEngine `vasic-digital-github` fork divergent
ISSUE-003	HXL-001	HelixLLM `analysis_test.go` hardcoded path
ISSUE-004	HXL-002	HelixLLM TOON `WriteTOON` 500
ISSUE-005	HXC-001	CONST-052 rename programme (project-wide)
ISSUE-006	HXC-002	Round-74 residual LOGIC FAILs (multi-submodule cross-cutting)
ISSUE-007	HXC-003	CONST-046 migration backlog (project-wide)
ISSUE-008	HXQ-001	helix_qa `TestPerformance` flake
ISSUE-009	HXA-001	helix_agent handler tests (4)
ISSUE-010	HXA-002	helix_agent debate API drift
ISSUE-011	HXA-003	venice CONST-037 (in helix_agent)

HXC-153 — TestGuard_GetSystemStatus_WithDB_StillReports is named for a database-present case it never exercises

Status: Queued **Type:** Bug **Severity:** Medium **Created-By:** Claude

A regression test in the server package is named as though it proves the system-status endpoint behaves correctly when a real database IS attached, but the test actually builds a server with no database at all and then only checks that the response code is 200. That means it silently re-runs the same no-database case a neighbouring test already covers, and the database-present path it advertises has no coverage from it whatsoever. This matters because the name is what a reader trusts when deciding whether a behaviour is guarded: anyone scanning the suite would reasonably conclude the with-database path is protected when it is not. It is the same documentation-versus-reality defect class as HXC-152, where a file's comment disagreed with its code, and it was surfaced by the independent

reviewer of that batch. The test's own comment does honestly defer coverage to another file, so nothing is being falsely asserted about the product — the defect is confined to the misleading name. Done means the test either is renamed to describe what it genuinely checks, or is given a fake-database seam so it earns the name it already has, with the choice backed by a captured run.

HXC-154 — TestStartQASession_Success asserts on a session field a background goroutine is concurrently changing

Status: Queued **Type:** Bug **Severity:** Medium **Created-By:** Claude

A QA-session test checks that a freshly started session reports the status 'pending', but the value it reads is one that a background worker is racing to change to 'running' at that very moment. The server hands the JSON encoder the same live session object the worker mutates, so whether the response says 'pending' or 'running' depends purely on which goroutine wins, and under load or with the race detector enabled the worker can win. The result is a test that usually passes and occasionally fails for reasons unrelated to any real product defect, which erodes trust in the suite and trains readers to dismiss red runs. It was observed failing once during unrelated work, then passed eleven consecutive full-package race runs afterwards, so it is genuinely rare rather than broken outright. The underlying product behaviour is not necessarily wrong — a caller may legitimately see either state — so the likely correct outcome is to make the assertion accept either value or to observe the status through a synchronised read instead. Done means the test no longer depends on goroutine scheduling, proven by repeated runs under the race detector, with the reproduction captured first.

HXC-155 — Standardise the divergent polarity-switch environment variable convention across the server test package

Status: Queued **Type:** Task **Severity:** Low **Created-By:** Claude

The server package's regression guards each support a switch that flips them between reproducing an old defect and guarding against its return, but the switch is spelled four different ways across the files and is read through three different helper styles. Having several names is genuinely useful, because each guard corresponds to a different historical fix and testers need to flip them one at a time rather than all at once, so the goal is not to collapse them into one variable. The problem is that the inconsistency has already caused real defects twice: one file shipped with its default set the wrong way round so its verification checks never

ran, and another shipped with a comment describing the opposite of what its code did. Both were fixed, but the drift that produced them remains. This work is to agree one naming pattern and one helper shape, apply it to every guard in the package, and record the convention in a single place so the next guard author copies something correct. Done means every switch follows the documented pattern, every default is the safe standing-guard direction, and a check exists that would catch a future file breaking the convention.

HXC-156 — harmony_os background system monitor used an unsynchronised on/off flag as its stop signal, racing application shutdown

Status: Queued **Type:** Bug **Severity:** High **Created-By:** Claude

The Harmony OS desktop application runs a background system monitor that samples processor and memory usage every few seconds. To decide whether to keep going, that monitor repeatedly read a plain on/off marker, while the application's shutdown routine switched the very same marker off from a different execution thread — with nothing coordinating the two. Two threads touching the same value at the same moment with no coordination is a data race: the shutdown signal can be missed entirely, leaving the monitor running and still writing to shared state after the application believes it has torn itself down, and on some machines the value read is not guaranteed to be either the old or the new one. This was not theoretical; it surfaced as a reproducible test failure, with the Harmony OS test suite failing on its cleanup test on every single run once race detection was enabled. To reproduce it, run the Harmony OS test package with the Go race detector switched on; the report names the monitor loop and the cleanup routine as the two conflicting parties. The same audit found a second, quieter instance of the same problem in the same component: the measured processor, memory, temperature and power figures were written by the monitor thread while the system-monitoring screen read them for display, again with no coordination, which could paint a screen mixing numbers from two different sampling rounds. The fix makes the monitor stop on the same shutdown channel the rest of the application already uses, protects the shared figures and the status marker with a lock, and makes shutdown wait for the monitor to confirm it has genuinely finished before proceeding. Acceptance: the Harmony OS test package completes with zero races over repeated consecutive runs under the race detector, and deliberately restoring the old code makes the identical race reappear, proving the guard is real and not merely agreeable.

HXC-157 — harmony_os and aurora_os changed on-screen elements directly from background threads, which the Fyne UI toolkit forbids

Status: Queued **Type:** Bug **Severity:** High **Created-By:** Claude

The Harmony OS and Aurora OS desktop front-ends both perform slow work in the background — asking a language model for an answer, polling provider health, refreshing dashboard statistics, starting the server — and each of those background workers wrote its result straight into an on-screen element. The user interface toolkit this product uses does not permit that: on-screen elements may only be changed from the single thread that draws them, and any other thread must hand its update over to be applied there. Doing it directly means the drawing code can be reading an element at the exact moment a background worker rewrites it, which is a data race that can corrupt what the user sees or crash the application outright. The problem was disguised rather than hidden: in both applications the offending line sat directly beneath a comment stating ‘Update UI on main thread’, which was simply untrue — it is the identical false comment already removed from the desktop front-end when this same defect class was fixed there. Two further aggravating factors were found by auditing every background worker in both files rather than only the reported line: several of these workers also read on-screen elements to decide what to do, which races just as surely as writing them, and several ran on endless timers with no way to stop, so they kept modifying elements belonging to a window that had already been closed and leaked for the lifetime of the process. The fix routes every such update through the shared dispatch helper introduced for the desktop front-end, keeps each read-and-then-write pair inside a single handover so the read cannot be left behind, moves the reads that happen at dispatch time onto the drawing thread where they belong, corrects the false comments in place so a later edit is not invited to undo the fix on their authority, and makes the endless timers stop when the application shuts down. Acceptance: both packages build and pass with the race detector enabled over repeated runs, no background worker in either file touches an on-screen element without going through the helper, and the desktop front-end’s behaviour is unchanged.

HXC-158 — No test builds the harmony_os or aurora_os screens, so their widget-threading fixes rest on code review rather than captured runtime proof

Status: Queued **Type:** Bug **Severity:** Medium **Created-By:** Claude

The Harmony OS and Aurora OS front-ends were just corrected so that background workers no longer modify on-screen elements directly. That correction is, however, only partly backed by evidence. No test in either package ever constructs the application’s screens, so the background workers that paint them are never started while the tests run, and the race detector therefore never gets the opportunity to observe the very code paths that were changed. Concretely, running the Aurora OS package under the race detector reported zero races both before and after the fix — not because the defect was absent, but because nothing under test reaches it; the corresponding Harmony OS failure that was reproduced and fixed came from a different component, the background system monitor, which tests do start. The practical risk is that a future edit could silently reintroduce a direct on-screen write from a background worker in either file and every test would stay green, exactly the situation the anti-bluff policy exists to prevent, since a green suite would then be certifying behaviour nobody has actually exercised. Closing this requires a test that builds the tabs and drives those workers — the chat worker, the provider-health poller and the dashboard and resource timers — with the race detector on, so the fix is proven by observation rather than by inspection, together with a deliberately-broken variant proving the new test genuinely fails when the direct write is put back. Acceptance: a test in each package starts the previously-unexercised background workers against real widgets, passes with zero races over repeated runs, and demonstrably fails when the dispatch helper is removed from any one of those sites. Until that exists, the fixes for those specific sites should be described as review-justified, not runtime-proven.